NautilusTRX

NautilusTRX is a financial tracker project designed by Jorge Valenzuela in spring 2026 for CIS 400.

This document summarizes each development pass and records lessons from it. Prompt excerpts needed for the course are included below; the original project repository is not required to use this handout.

Table of Contents

`milestone-instructions`

In `milestone-instructions` the original html instructions posted on canvas have been converted to Markdown for easier use in agentic coding test projects. The content of the files have been preserved, save for images and several canvas internal links.

There are three main sections for development - Background Logic -> Milestone 0 - 4 - WPF GUI App -> Milestone 5 - 9 - Web App (Razor Page) -> Milestone 10 - 12

`starting-code`

This contains the starting code for the project (i.e. ready for milestone 0).

`first-pass`

In the first pass of building the project, the goal was discover what a project like this could teach me about agentic development.

In `first-pass/milestones`, I used unaltered instructions. The extra information in the specs that didn't seem useful to Claude seems to have little impact on the project. However it is still extra tokens being used that do not contribute to the project.

Tools I used: (normally not checked into source control, but for this project I keep them in.)

- `grill-me` for requirement elicitation
- `transpose` skill for recording my conversation with claude.

`second-pass`

In the second pass, I took the tooling suggestions from the first and put them to use (starting with the CLAUDE.md). I keep with the same un-altered spec files and followed a similar process, focused this time to lean on the tools I made, and being continuous on cost.

Tools I used: (normally not checked into source control, but for this project I keep them in.)

- `grill-me` for requirement elicitation
- `transpose` still and hook for recording my conversation with claude (using Haiku).
- cost recording tools. Cost logged in a CSV and I added a new status line to show the in-progress cost of developments.

I will use `opusplan` model as default. Since I expect to have highly rigorous plans backed by spec files by the time we implement, Opus will be used for planning and Sonnet will be used for implementing.

`third-pass`

In the third pass, I took a different approach. Rather than using the milestone information to work on the project, I generated a ConOps from the milestone instructions and used that to generate the entire project.

I started with a base project, but zero code this time. I ran `/init` to initialize the project and added some guidelines and conventions to follow. (C#, Data, WPF app Razor Page app, git conventions)

I then ran the following prompt (in plan mode) to generate the project wide plan.

```
@Documentation/ConOps.md is the ConOps for the project. Use this to generate a
comprehensive development plan. Organize it into natural waves of development, or
points where we can review the work before moving forward.
```

A strong plan that was split up into waves was generated. I added a link to the plan in the markdown so Claude could always reference the plan. The waves were as follows

0. Guardrails & baseline (small, fast checkpoint)
1. Transaction core (`Data/Enums`, `Data/Transactions`)
2. Account & current balance (`Data/Accounts`)
3. Ledger, net worth & financial summary (`Data/Ledger.cs`)
4. Payments & confirmations (`Data/Payments`)
5. Statements (`Data/Statements`)
6. WPF desktop app (`WpfApp` → promote to root `NautilusTrX.sln`)
7. Website (`Website` = `NautilusTrX.Website`, ASP.NET Core Razor Pages)
8. Final integration & polish

When comparing to the milestones, while not dissimilar, it is hard to pull a strong relationship between the two sets of instructions and steps.

One learning after the project was complete as to **look out for waves that were to large in scope**. Both milestone 6 and 7 were much to large and should have been split at the start of planning.

`fourth-pass`

I used Beads for this fourth pass. Beads is likely a very useful application, but it currently requires more experience with issue tracker, larger projects, or lots of time to really learn how to use it effectively.

Below are the epics and tasks produced in Plan mode. The excerpt is self-contained; the full project plan and prompt log are not bundled with this handout.

> **The epics (beads graph)** Dependencies: **E0 → E1 → E2; E2 → E3 and E2 → E4 (parallel); E3,E4 → E5.**
>
> - E0 — Guardrails & scaffolding
> - E1 — Domain core (model + derived position)
> - E2 — Payments & statements (domain services)
> - E3 — Desktop WPF (MVVM) + FlaUI verification *(depends on E2)*
> - E4 — Web Razor Pages + Playwright verification
>
> - E5 — Integration sweep & docs *(depends on E3, E4)*

On approving this plan, it added the tasks to the beads tracker, then started implementing and didn't stop until it was 100% done.

Learning: In this process I enforced stronger testing requirement as I knew the use of Beads would likely make it more autonomous. By giving it all the tools it needed to check itself, it was better at finding bugs.

However, I failed to get it to pause for me to check it. In the future, I should have interrupted the model as soon as it skipped a verification step I intended.

As for economics, I ran the entire thing in one session of Opus, and despite the lack of new sessions, the entire project came out to be 25—50 in API pricing. A large reason for that is there was no additional grilling or re-contextualizing after a milestone.

The overall quality of the final product was well below par. Since I didn't verify the correct direction early on, it would require an expensive review to find and correct the organizational and stylistic choices Claude made.

`fifth-pass`

The fifth pass is a polished and intentional implementation of the NautilusTrX project. It uses the following Agentic design principles as guiding philosophies for every decision.

- Spec Driven Development
- The Cycle of Development
- Project Context Management
- Requirement Elicitation
- Verification

Extra care was taken to ensure no previous pass influenced this pass. (Permission deny in a settings.json. This would normally be a local only file, but kept in for documentation purposes)

For this project, I only used the Opus model.

Development Plan

The fifth-pass development plan was generated from an initial grilling session that split the project ConOps into waves.

It included 10 waves of development:

0. Foundation & test harness
1. Domain: Accounts & Transactions model
2. Domain: Position, Summaries & Overdue
3. Domain: Payments & Records
4. Desktop: shell + Accounts
5. Desktop: Transactions & dashboard
6. Desktop: Payments & Statements
7. Web: shell + read views
8. Web: search & filter (URL)
9. Web: transaction form & file persistence

Each wave of development would start with the following prompt (normal mode):

```
Implement Wave 2. /grill-me
```

Additional information was also added if I knew I already had a design decision made that it didn't know about.

After grilling was complete, it followed the following development cycle:

- ☐ **Red** — write the wave's tests first (primarily from the ConOps; extra tests allowed), enumerating every branch listed for the wave; confirm they fail / don't compile.
- ☐ **Green** — implement the minimum to pass, preferring manual/simple code.
- ☐ **Coverage gate** — `dotnet test` green; coverage projects at 100% branch; generate the ReportGenerator HTML and confirm no uncovered branches (or documented exclusions).
- ☐ **Self review** — run a `code-review` subagent over the wave diff; address findings.

☐ **Manual checklist** — output a checklist of what was implemented for the user's review.
☐ **Checkoff** — wait for the user's approval.
☐ **Commit, push, `/transcribe`** the session.

Learnings and Final Product

I prompted Claude to review my use of the development practices. That review was insightful, but different from my own review of the project.

I intentionally spent more time (8-9 hrs) on this pass than the previous passes. This resulted in a high quality product I would be satisfied in presenting to my company or turning in for a project.

However there are still weaknesses. Most importantly is (and will be for the foreseeable future) that **I can only provide council on what I already know**. There were several software engineering principles used by Claude that I was not as familiar with (seams for testing, separate guards class, most of cshtml) so when it came to requirement elicitation (`/grill-me`) and verification, I sometimes made the wrong assumptions of what it was talking about and had to take extra time during development to either change what the plan was, or to convince myself that it made sense.

That being said, the application and use of the 5 development principles were clear throughout the project and helped produce a better result.

- **Spec Driven Development**
  1. From the ConOps, create an implementation plan. Use this plan and the ConOps as reference documents for the rest of the project.
- **The Cycle of Development**
  1. The Cycle was hard written into the plan and enforced each time Claude began writing code. (Tip: The tool **TodoWrite** will create a checklist for Claude to follow. Very useful to enforce a Cycle of Development)
- **Project Context Management**
  1. Conventions and preferences added before Wave 1 started
  2. Upkeep of the current state of the project in CLAUDE.md
  3. Attempted to enforce the cycle of development when it wasn't starting the checklist (failed because I didn't know the name of the tool)
  4. Added Style guidelines before GUI and Website development started
- **Requirement Elicitation**
  1. Extensive and uniform use of `/grill-me` skill across the project. Both in initial planning and before each wave.
- **Verification**
  1. Baked into the Cycle of development.
  2. Organized the project to allow 100% branch coverage of logic code
  3. Included tools so Claude and run and verify both branch coverage and GUI Behavior
  4. Manual review step to look at code and test GUI for myself.

Personal satisfaction: **8/10** Best end product and best understanding of the end product so far.